

Mathematical Description of the Unicorn XJ Tag

Julian Regalado Perez
julian.perez@sund.ku.dk

1 Prompt

Ok, I need you to provide me a latex file containing a thorough description of the math behind the XJ tag alignment score recomputation.

Remember the implementation is found in `unicorn_internal.c::unicorn_addjscore()`.

Add a small overall intro, an intro to the notation followed by method description.

At the end add a working example with an alignment of a damaged fragment and another one with an undamaged fragment.

Add the queries explicitly. Include both 5' C-T and 3' G-A. Place the document in `XJtag.tex`.

2 Overview

The Unicorn XJ tag is a damage-aware floating point alignment penalty computed by the C function

`unicorn_addjscore()`

in `src/libunicorn/unicorn_internal.c`. The purpose of the tag is to recompute the mismatch component of an alignment score in a way that is less punitive for substitutions that are characteristic of ancient DNA damage: 5' C-to-T substitutions and 3' G-to-A substitutions.

The implementation should be read as a penalty score rather than a reward score. Lower values indicate either fewer discrepancies or discrepancies that are more compatible with terminal damage. Ordinary mismatches receive a penalty of 1, while damage-compatible mismatches receive a penalty between 0 and 1 depending on their distance from the relevant fragment end. Insertions and deletions are penalized with affine-like gap constants supplied by the caller. In the current Unicorn callers, those constants are

$$g_o = 5, \quad g_e = 2,$$

where g_o is the gap opening penalty and g_e is the per-base gap extension penalty.

The final value is stored as a SAM/BAM auxiliary tag

`XJ:f:<score>`.

If an XJ tag already exists, it is deleted and replaced by the newly computed value.

3 Notation

Consider one aligned BAM record. Let

$$Q = Q_0 Q_1 \cdots Q_{L-1}$$

be the query sequence stored in the BAM record, with query length L . Query positions are zero-based, so Q_0 is the first base and Q_{L-1} is the last base. The implementation uses the query coordinates and query bases exactly as provided by `bam_get_seq()` and `bam_seqi()`.

The alignment is described by two pieces of SAM/BAM information:

- The CIGAR string, which describes how query columns and reference columns are related.
- The MD tag, which gives the reference bases at mismatch positions and the reference bases deleted from the alignment.

Let r denote a reference-column index in the alignment coordinate system used internally by the function. The implementation builds an array

$$\text{ref2q}(r)$$

that maps each reference-consuming alignment column to a query position:

$$\text{ref2q}(r) = \begin{cases} q, & \text{if reference column } r \text{ aligns to query base } Q_q, \\ -1, & \text{if reference column } r \text{ is a deletion from the query,} \\ -2, & \text{if reference column } r \text{ is a reference skip.} \end{cases}$$

Only mismatch positions with $\text{ref2q}(r) \geq 0$ contribute mismatch penalties. Deleted reference bases are handled by the gap term, not by the mismatch term.

The default damage model constants in the implementation are

$$\alpha = 1.0, \quad \lambda = 5.0,$$

where α is the ordinary mismatch penalty and λ controls the exponential decay of the terminal damage discount.

4 Method

4.1 Gap contribution

The function first scans the CIGAR string. Insertions and deletions contribute gap penalties. For an insertion of length ℓ ,

$$S_I(\ell) = g_o + g_e \ell.$$

For a deletion of length ℓ ,

$$S_D(\ell) = g_o + g_e \ell.$$

Thus the total gap contribution is

$$S_{\text{gap}} = \sum_{I \in \mathcal{I}} (g_o + g_e \ell_I) + \sum_{D \in \mathcal{D}} (g_o + g_e \ell_D),$$

where $\mathcal{I}$ is the set of CIGAR insertion operations and $\mathcal{D}$ is the set of CIGAR deletion operations.

Soft clips advance the query coordinate but do not add a penalty. Reference skips consume reference columns but do not add a penalty. Matches, sequence matches, and sequence mismatches consume both reference and query columns but do not directly add a gap penalty.

4.2 Mismatch contribution

Mismatch positions are recovered by parsing the MD tag. When the MD parser sees a number, it advances by that many matching reference columns. When it sees a deletion block beginning with $\sim$, it advances the reference coordinate through the deleted bases. When it sees an alphabetic base, that base is the reference base at a mismatch position.

For a mismatch at reference column r , let

$$q = \text{ref2q}(r)$$

be the aligned query coordinate. Let R_r be the reference base from the MD tag and let Q_q be the query base from the BAM sequence. If either base is N, the mismatch is ignored by the implementation.

The damage probability assigned to the mismatch is

$$p_{\text{damage}}(R_r, Q_q, q, L) = \begin{cases} \exp(-q/\lambda), & \text{if } R_r = \text{C and } Q_q = \text{T}, \\ \exp(-(L - q - 1)/\lambda), & \text{if } R_r = \text{G and } Q_q = \text{A}, \\ 0, & \text{otherwise.} \end{cases}$$

This means that C-to-T mismatches are discounted most strongly near the 5' end, while G-to-A mismatches are discounted most strongly near the 3' end. The implementation clamps this probability to the interval $[0, 1]$ after computing it.

The mismatch penalty is then

$$S_{\text{mm}}(R_r, Q_q, q, L) = \alpha (1 - p_{\text{damage}}(R_r, Q_q, q, L)).$$

With $\alpha = 1$, ordinary mismatches have penalty 1. A terminal damage-compatible mismatch can have penalty 0, because $\exp(0) = 1$ at the relevant end.

4.3 Final XJ value

Let $\mathcal{M}$ be the set of mismatch positions reported by the MD tag that map to real query bases and do not contain N. The final score stored in the XJ tag is

$$XJ = S_{\text{gap}} + \sum_{r \in \mathcal{M}} S_{\text{mm}}(R_r, Q_{\text{ref2q}(r)}, \text{ref2q}(r), L).$$

The function does not use the original aligner score, the AS tag, or the edit distance tag NM. The XJ value is recomputed from the CIGAR string, the query sequence, and the MD tag. If the BAM record is missing an MD tag, no new XJ tag is written.

5 Working Examples

In both examples below, the CIGAR string is 16M, so there are no insertions or deletions and

$$S_{\text{gap}} = 0.$$

The query length is $L = 16$, the default damage decay length is $\lambda = 5$, and the ordinary mismatch penalty is $\alpha = 1$.

5.1 Damaged fragment with 5-prime C-to-T and 3-prime G-to-A

Reference:

ACGTACCGTACGTAGC

Query:

ATGTACCGTACGTAAC

Alignment summary:

Position q	Reference base	Query base	Damage class	Penalty
1	C	T	5' C-to-T	$1 - \exp(-1/5)$
14	G	A	3' G-to-A	$1 - \exp(-(16 - 14 - 1)/5)$

The corresponding MD tag is

MD:Z:1C12G1.

It means: one matching base, then a reference C mismatch, then twelve matching bases, then a reference G mismatch, then one final matching base.

For the 5' C-to-T mismatch at $q = 1$,

$$S_{\text{mm}} = 1 - \exp(-1/5) = 1 - \exp(-0.2) \approx 0.18127.$$

For the 3' G-to-A mismatch at $q = 14$, the distance from the 3' end is

$$L - q - 1 = 16 - 14 - 1 = 1,$$

so

$$S_{\text{mm}} = 1 - \exp(-1/5) \approx 0.18127.$$

Therefore

$$XJ = 0 + 0.18127 + 0.18127 \approx 0.36254.$$

The resulting tag would be approximately

XJ:f:0.36254.

5.2 Undamaged fragment with an ordinary internal mismatch

Reference:

ACGTACCGTACGTAGC

Query:

ACGTGCCGTACGTAGC

This query has one internal A-to-G mismatch at query position $q = 4$. It is not a C-to-T mismatch and it is not a G-to-A mismatch, so it receives no damage discount.

The corresponding MD tag is

MD:Z:4A11.

It means: four matching bases, then a reference A mismatch, then eleven matching bases.

For this mismatch,

$$p_{\text{damage}} = 0,$$

and therefore

$$S_{\text{mm}} = 1(1 - 0) = 1.$$

Because there are no gaps,

$$XJ = 0 + 1 = 1.$$

The resulting tag would be

$$\mathbf{XJ:f:1}.$$

If the undamaged query were an exact match to the reference,

$$\mathbf{ACGTACCGTACGTAGC},$$

with $\mathbf{MD:Z:16}$, then there would be no mismatch contribution and no gap contribution, so $XJ = 0$.